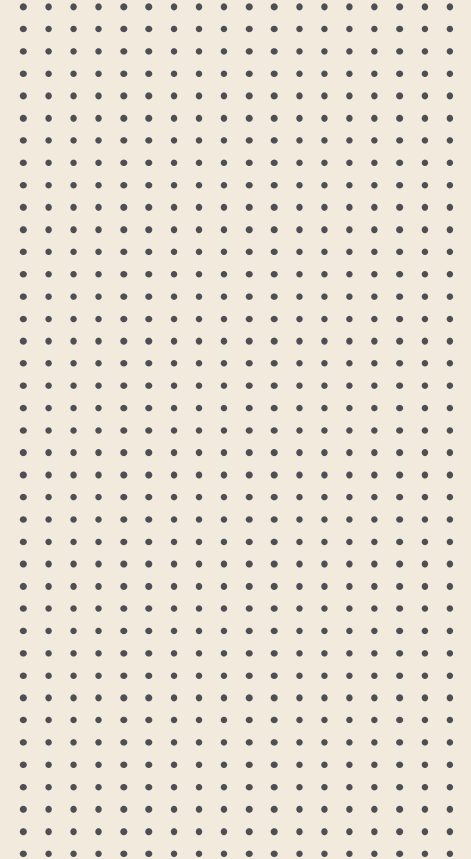


CSE-1223

Discrete Mathematics

Segment 8: Tree

Md. Saiful Islam
Lecturer, Dept. of CSE, IIUC





Tree

1

Definition of Trees

2

Application of trees

3

Tree Traversal

4

Trees and sorting

5

Spanning trees

6

Minimum spanning
trees

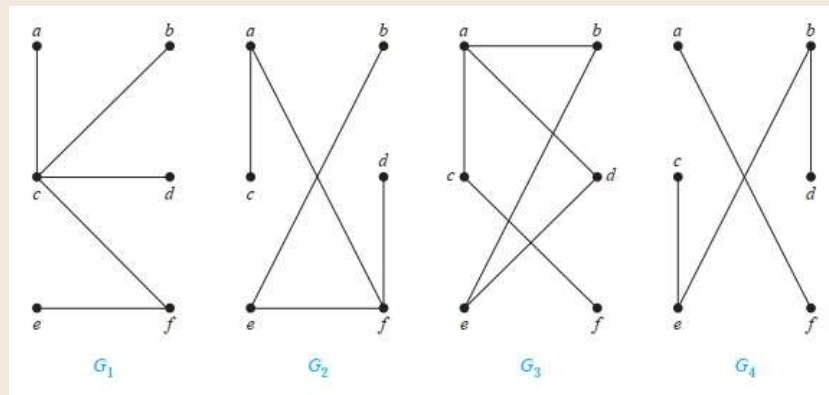
1

Definition of Trees

Definition of Trees

What is a **tree**? [A]

Ans: A **tree** is a connected undirected graph with no simple circuits.



Which of the graphs are trees?

Ans: G_1 and G_2 are trees, because both are connected graphs with no simple circuits. G_3 is not a tree because e, b, a, d, e is a simple circuit in this graph. Finally, G_4 is not a tree because it is not connected.



Definition of Trees



What is a **rooted tree**? [A]

Ans: A **rooted tree** is a tree in which one vertex has been designated as the root and every edge is directed away from the root.



Definition of Trees



Explain why every tree is a bipartite graph? [Aut'22]

Ans: Given that, T is a tree.

To prove: T is bipartite.

Let V_1 contains all vertices at level 0, level 2, level 4, etc., of the tree T and let V_2 contains all vertices at level 1, level 3, level 5, etc.,

There cannot be edges from a vertex in V_2 to a vertex in V_1 , as a vertex at level h can only be connected to a vertex at level $h-1$ or level $h+1$.

Similarly, there cannot be edges from a vertex in V_2 to a vertex in V_2 , as a vertex at level h can only be connected to a vertex at level $h-1$ or level $h+1$.

Finally, it is possible that there are edges from a vertex in V_1 to a vertex in V_2 .

Conclusion: So, the tree is bipartite.

Definition of Trees

Write down the differences between Graph and Tree? [[Aut'22](#)]

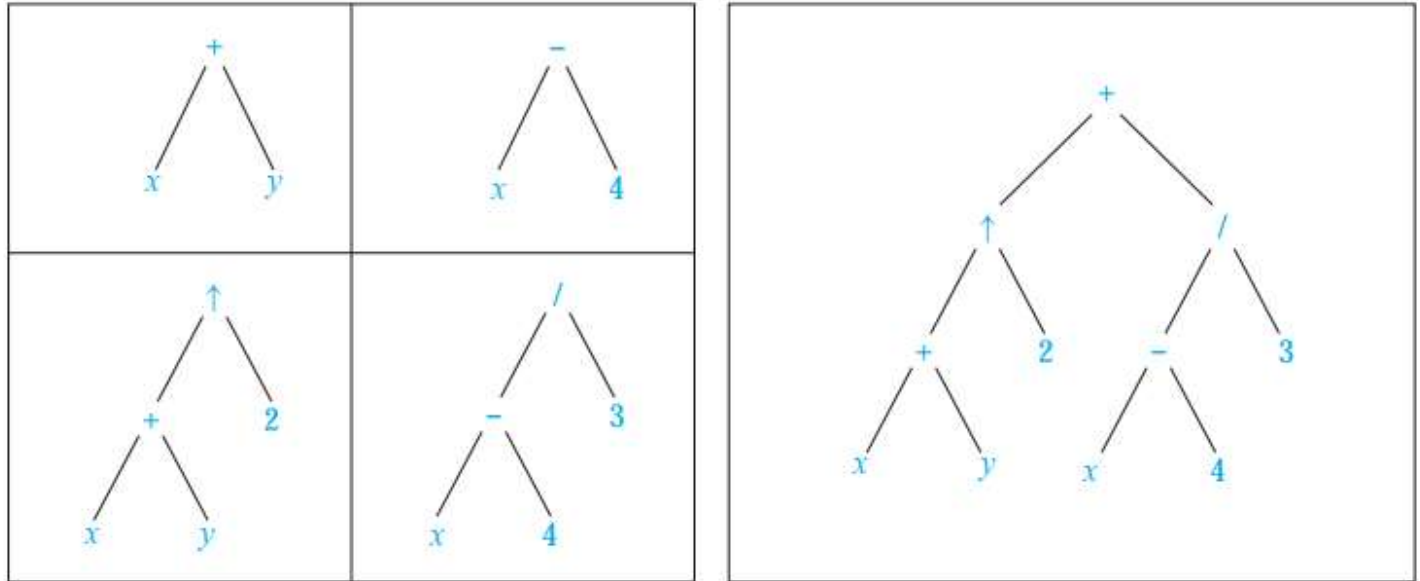
Ans:

The basis of Comparison	Graph	Tree
Edges	Each node can have any number of edges.	If there is n nodes then there would be n-1 number of edges
Types of Edges	They can be directed or undirected	They are always directed
Root node	There is no unique node called root in graph.	There is a unique node called root(parent) node in trees.
Loop Formation	A cycle can be formed.	There will not be any cycle.
Traversal	For graph traversal, we use Breadth-First Search (BFS), and Depth-First Search (DFS).	We traverse a tree using in-order, pre-order, or post-order traversal methods.
Applications	For finding shortest path in networking graph is used.	For game trees, decision trees, the tree is used.

Definition of Trees

What is the ordered rooted tree that represents the expression $((x + y)^2) + ((x - 4)/3)$?

Ans:



Definition of Trees

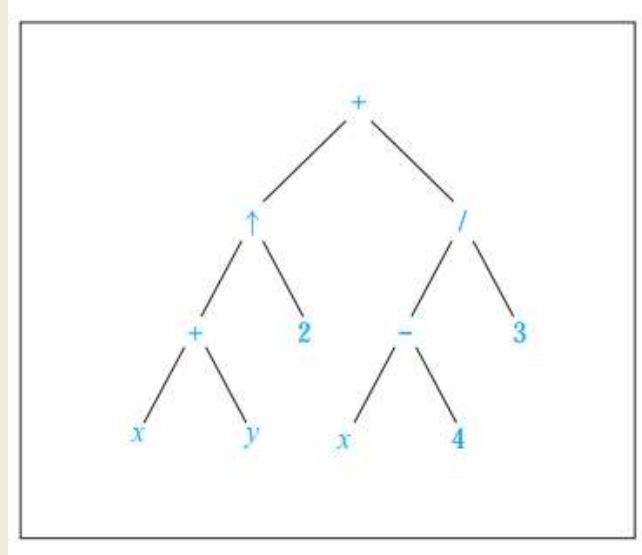
What is the prefix form for $((x + y) \uparrow 2) + ((x - 4)/3)$?

Ans: We obtain the prefix form for this expression by traversing the binary tree that represents it in preorder. This produces

$+ \uparrow + x y 2 / - x 4 3$

Exercise: Build a binary tree, prefix and postfix notation of the following expression:

$((5+z)/-8)*(4^2)$ [Aut'18]



2

Applications of Trees



Applications of Trees



What are the common **applications** of trees?

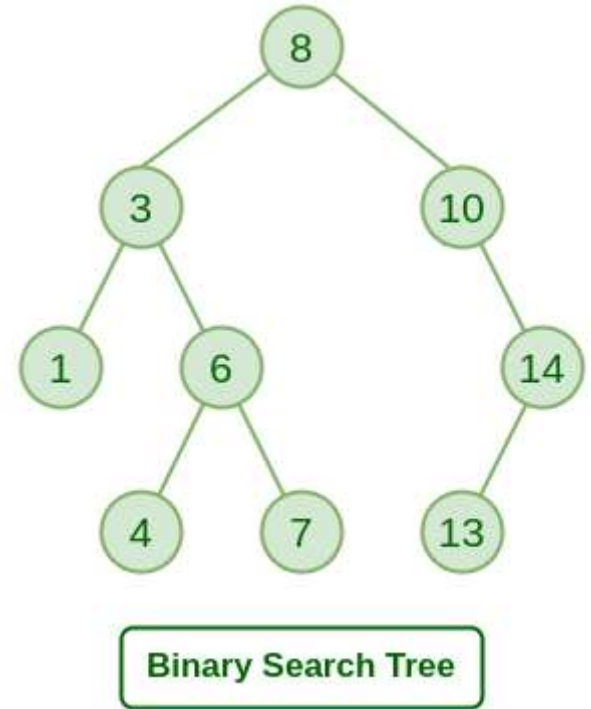
Ans: Trees find applications in various fields, including file systems, databases, network routing, artificial intelligence, syntax parsing, and more. They are used to organize and optimize data storage and retrieval, implement efficient searching algorithms, and represent relationships between entities.

Binary Search tree

What is **Binary Search tree**?

Ans: **Binary Search Tree** is a node-based binary tree data structure which has the following properties:

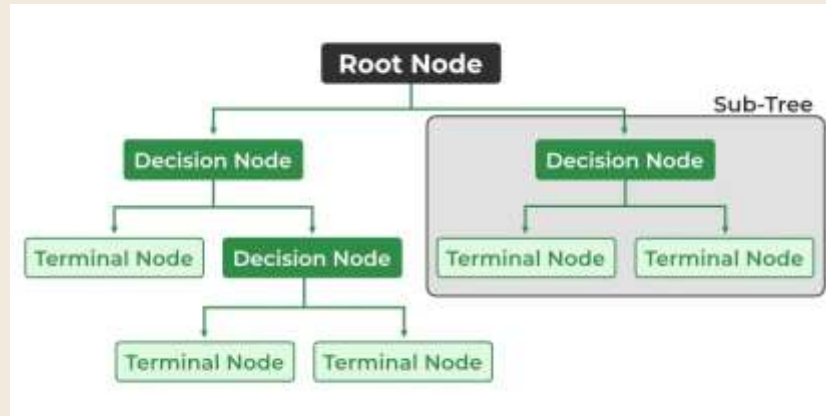
- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.



Decision tree

What is **Decision tree**?

Ans: A **decision tree** is a flowchart-like tree structure where each internal node denotes the feature, branches denote the rules and the leaf nodes denote the result of the algorithm. It is a versatile supervised machine-learning algorithm, which is used for both classification and regression problems. It is one of the very powerful algorithms.



3

Tree Traversal



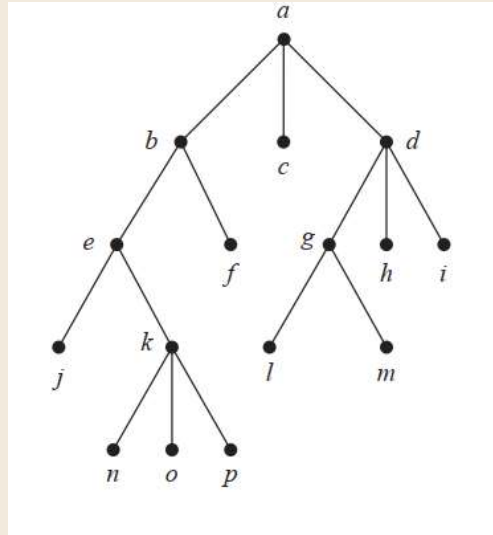
Tree Traversal

What do you mean by **tree traversing**? [Sp'23]

Ans: Procedures for systematically visiting every vertex of an ordered rooted tree are called **traversal** algorithms/tree traversing. We will describe three of the most commonly used such algorithms, **preorder** traversal, **inorder** traversal, and **postorder** traversal.

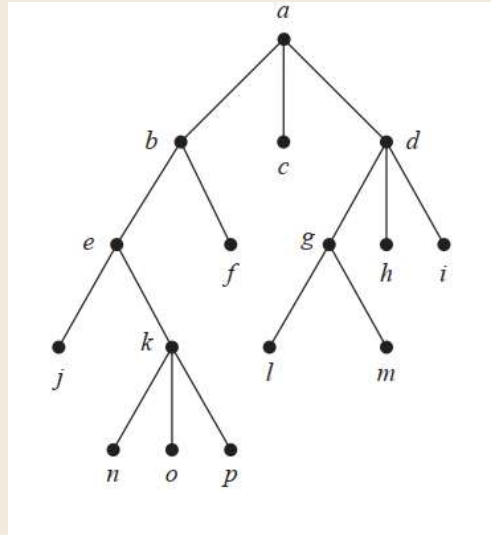
Tree Traversal | Preorder Traversal

Find the **preorder** traversal of the following ordered tree.

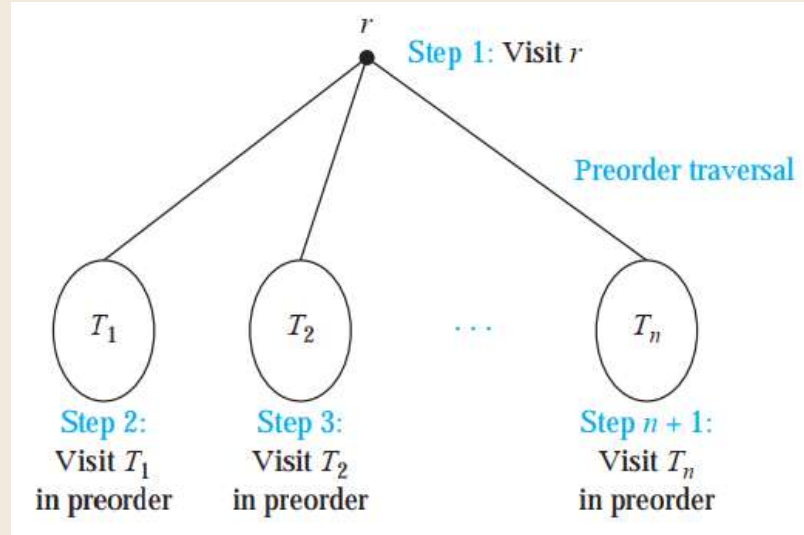


Tree Traversal | Preorder Traversal

Find the **preorder** traversal of the following ordered tree.

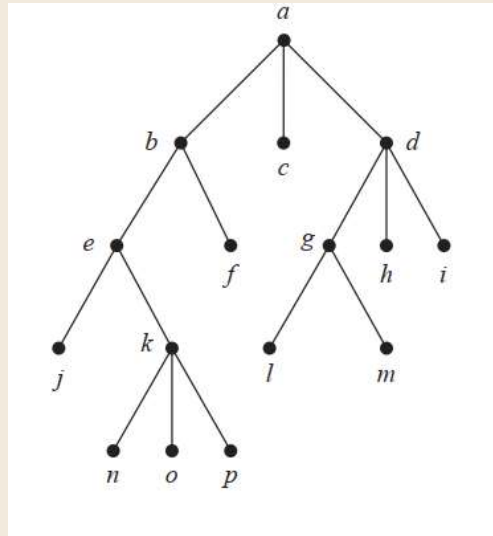


Ans:



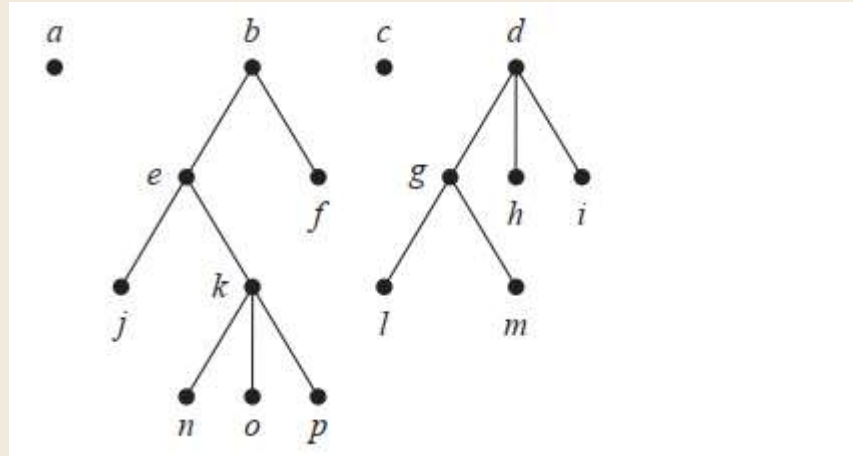
Tree Traversal | Preorder Traversal

Find the **preorder** traversal of the following ordered tree.



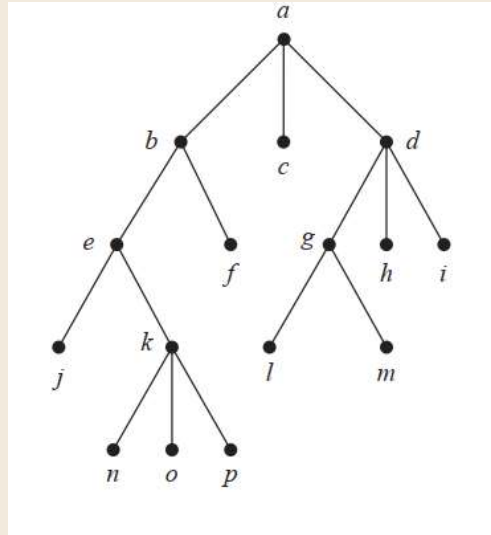
Ans: Preorder traversal: Visit root, visit subtrees left to right

Step 1:

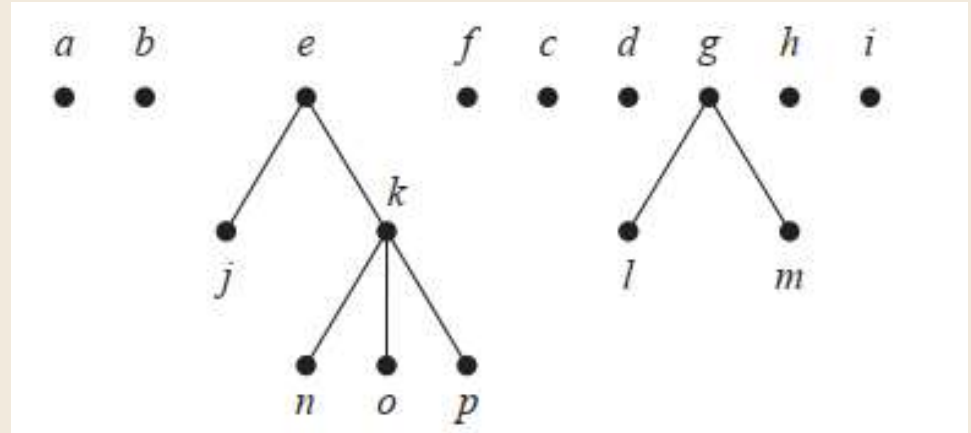


Tree Traversal | Preorder Traversal

Find the **preorder** traversal of the following ordered tree.

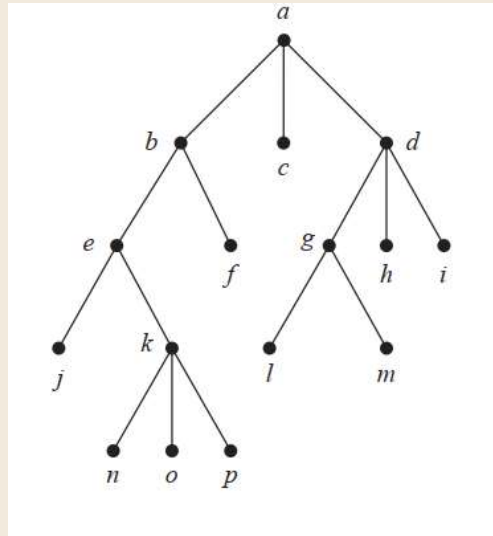


Ans: Step 2:

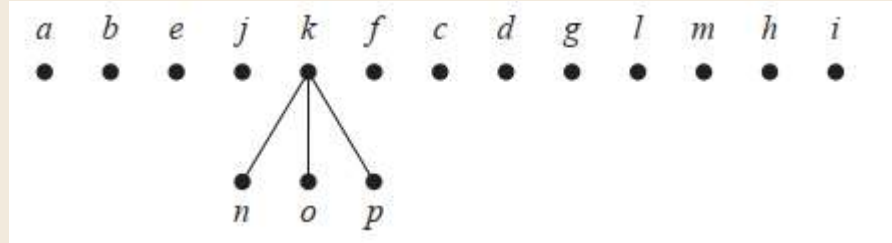


Tree Traversal | Preorder Traversal

Find the **preorder** traversal of the following ordered tree.

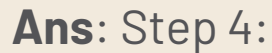


Ans: Step 3:



A 6x4 grid of dots, consisting of 6 rows and 4 columns of small black dots.

A 6x4 grid of dots, consisting of 6 rows and 4 columns of small black dots.



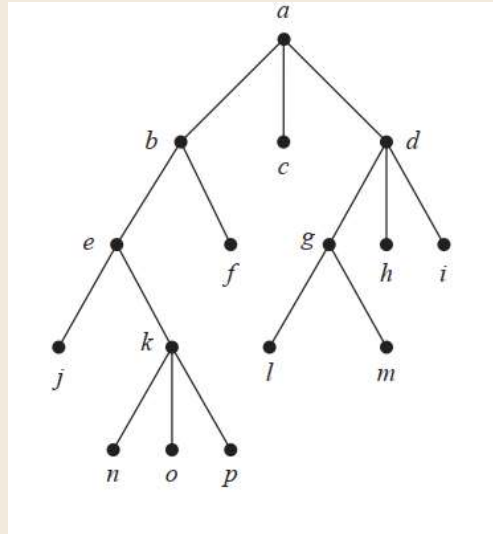
a b e j k n o p f c d g l m h i
● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ●



Tree Traversal | Inorder Traversal

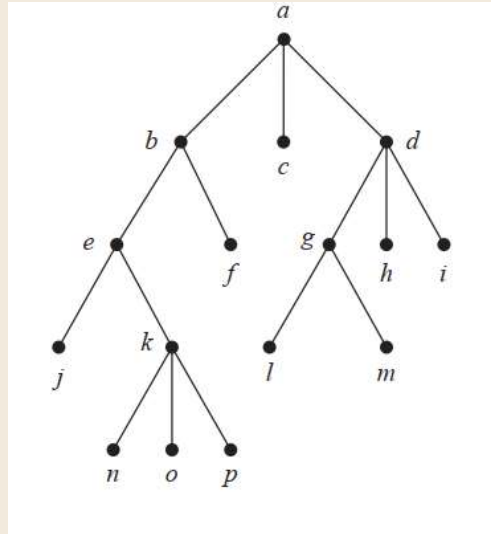


Find the **inorder** traversal of the following ordered tree.

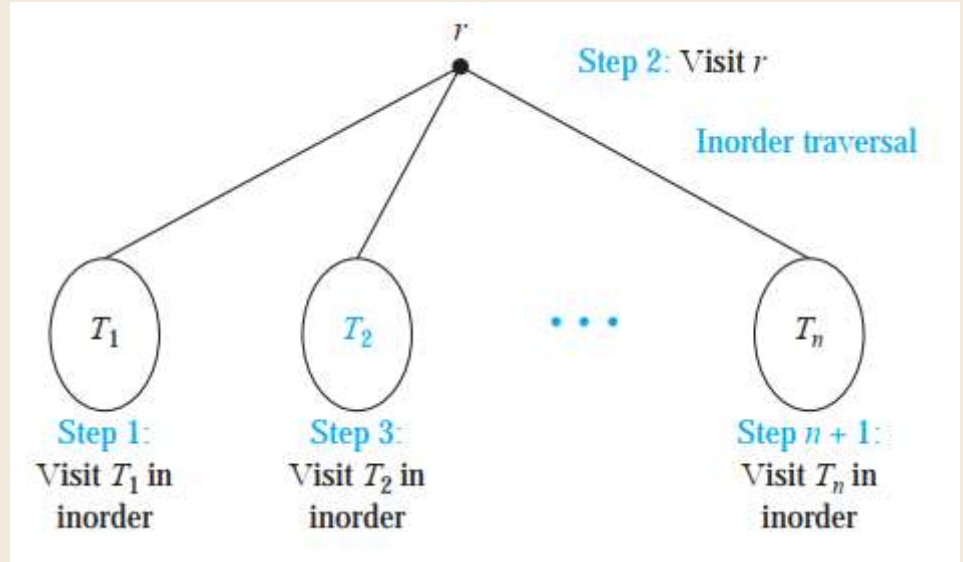


Tree Traversal | Inorder Traversal

Find the **inorder** traversal of the following ordered tree.

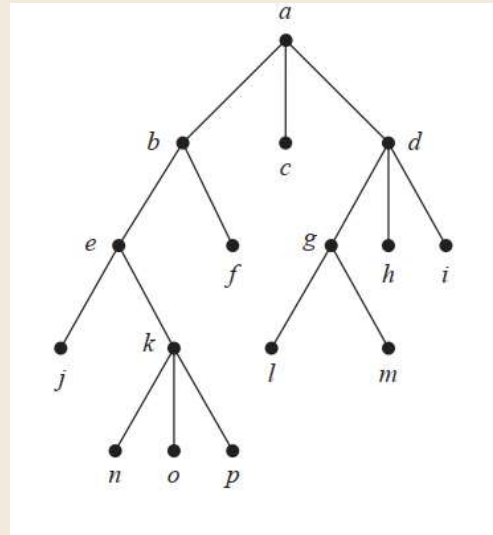


Ans:

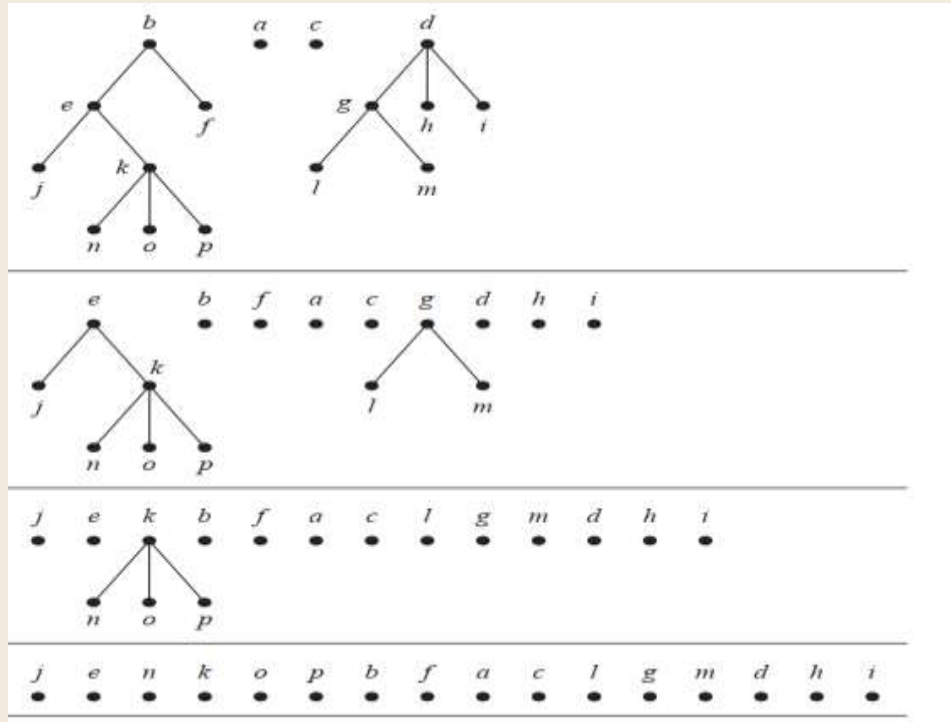


Tree Traversal | Inorder Traversal

Find the **inorder** traversal of the following ordered tree.

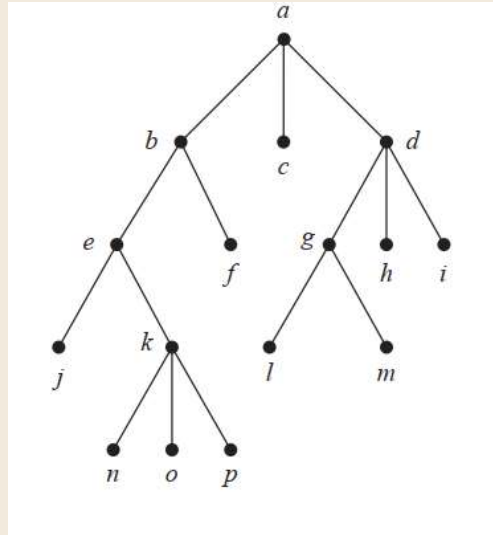


Ans:



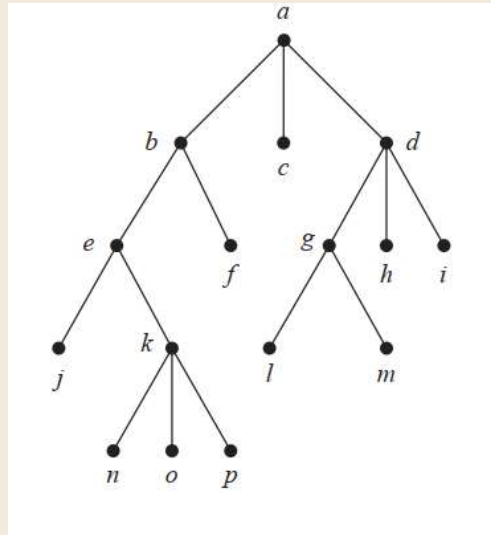
Tree Traversal | Postorder Traversal

Find the **postorder** traversal of the following ordered tree.

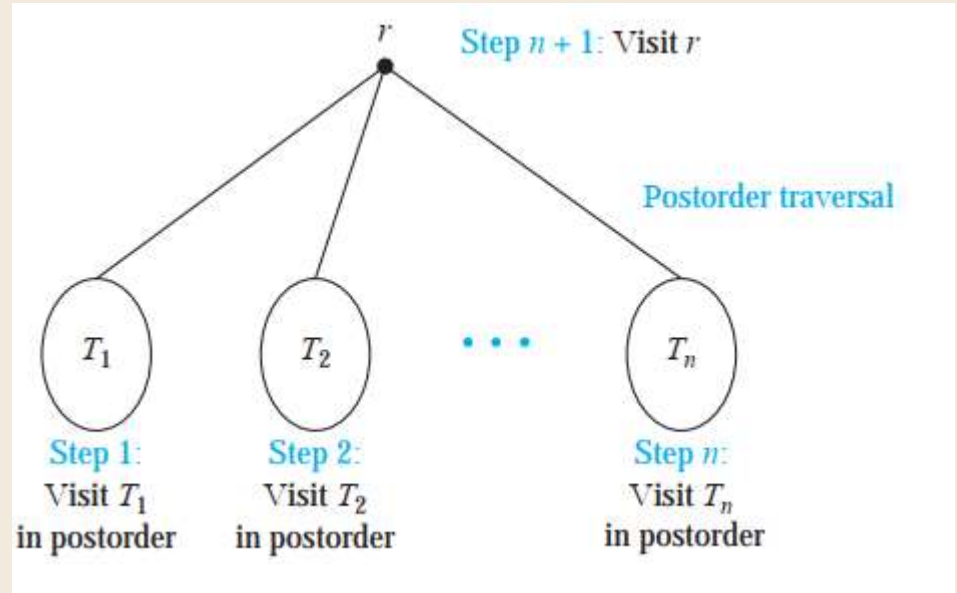


Tree Traversal | Postorder Traversal

Find the **postorder** traversal of the following ordered tree.

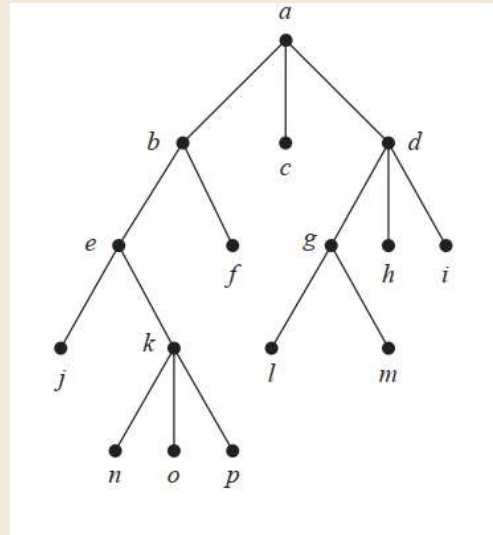


Ans:

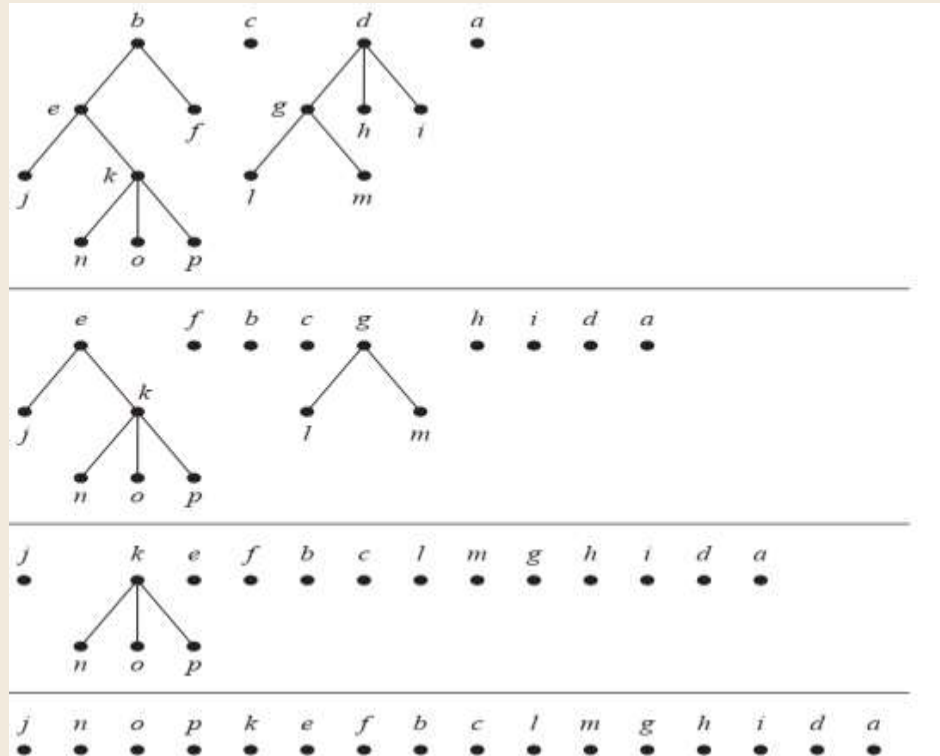


Tree Traversal | Postorder Traversal

Find the **postorder** traversal of the following ordered tree.



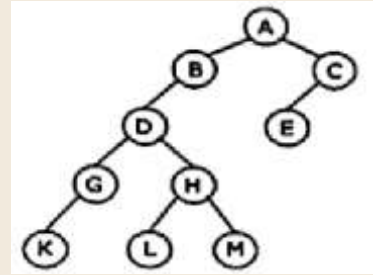
Ans:



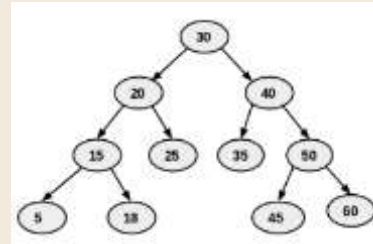
Tree Traversal

Exercise:

1. Find the **preorder**, **inorder** and **postorder** traversal of the following ordered tree. [[Sp'19,'22, Aut'20](#)]



2. Find the **preorder**, **inorder** and **postorder** traversal of the following ordered tree. [[Sp'23](#)]



5

Spanning Trees



Spanning Tree



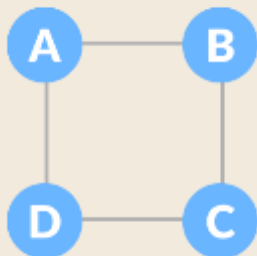
Define spanning tree. [Sp'22, Sp'19]

Ans: A **spanning tree** is a sub-graph of an undirected connected graph, which includes all the vertices of the graph with a minimum possible number of edges. If a vertex is missed, then it is not a spanning tree.

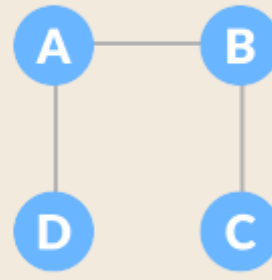
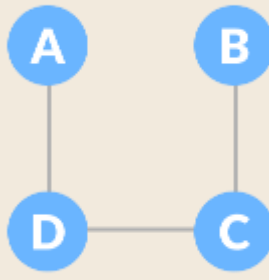
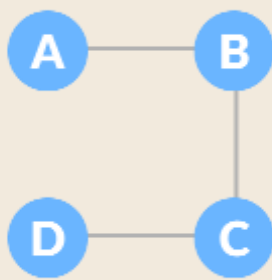
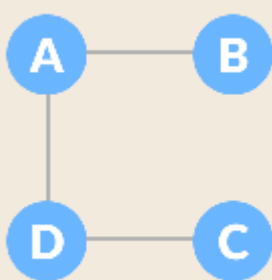


Spanning Tree

Find the spanning trees of the following graph:



Ans: Some spanning trees of the graph are:



Spanning Tree

Find a **spanning tree** of the simple graph G shown in Figure 2.

Ans: The graph G is connected, but it is not a tree because it contains simple circuits. **Remove the edge $\{a, e\}$.** This eliminates one simple circuit, and the resulting subgraph is still connected and still contains every vertex of G . Next **remove the edge $\{e, f\}$** to eliminate a second simple circuit. Finally, **remove edge $\{c, g\}$** to produce a simple graph with no simple circuits. This subgraph is a **spanning tree** because it is a tree that contains every vertex of G . The sequence of edge removals used to produce the spanning tree is illustrated in Figure 3.

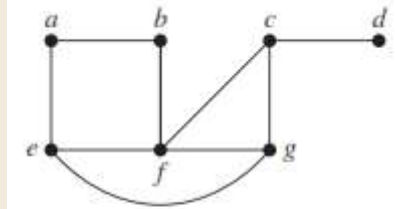
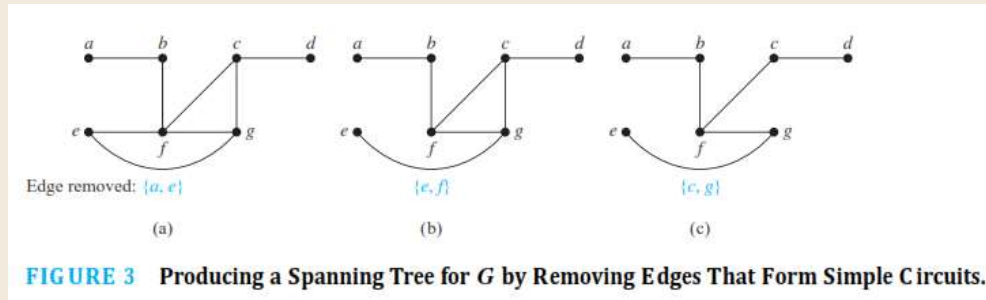


FIGURE 2 The Simple Graph G .



Spanning Tree

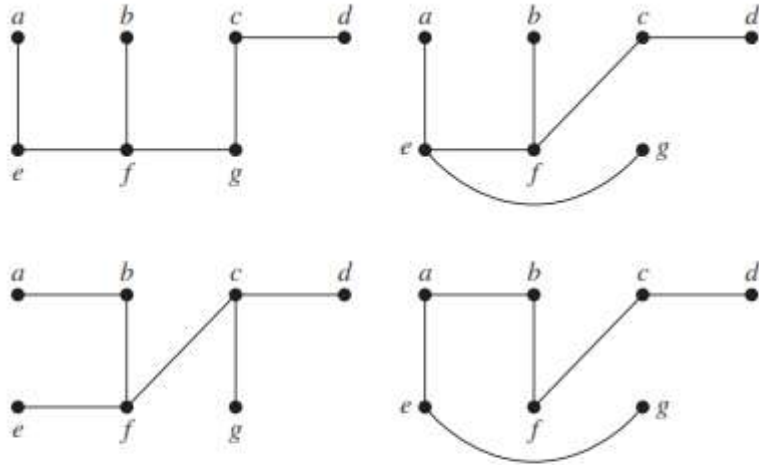


FIGURE 4 Spanning Trees of G .

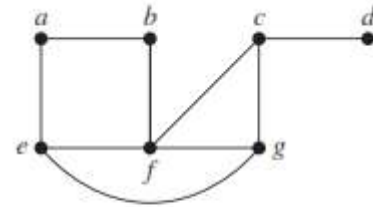


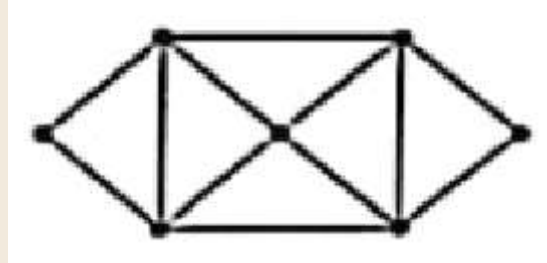
FIGURE 2 The Simple Graph G .

The tree shown in Figure 3 is not the only spanning tree of G . For instance, each of the trees shown in Figure 4 is a spanning tree of G . ▶

Spanning Tree

Exercise:

a) Find two different spanning trees of the following graph: [Aut'22]





Depth-First Search

Depth-first Search or Depth-first traversal is a recursive algorithm for searching all the vertices of a graph or tree data structure. Traversal means visiting all the nodes of a graph.

A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.



Depth-First Search



The DFS algorithm works as follows:

- Start by putting any one of the graph's vertices on top of a **stack**.
- Take the top item of the stack and add it to the visited list.
- Create a list of that vertex's adjacent nodes. Add the ones that aren't in the visited list to the top of the stack.
- Keep repeating steps 2 and 3 until the stack is empty.

Depth-First Search

Use **depth-first search** to find a spanning tree for graph G shown in Figure 6.

Ans: The steps used by depth-first search to produce a spanning tree of G are shown in Figure 7. We arbitrarily start with the vertex f. A path is built by successively adding edges incident with vertices not already in the path, as long as this is possible. This produces a path f, g, h, k, j (note that other paths could have been built). Next, backtrack to k. There is no path beginning at k containing vertices not already visited. So we backtrack to h. Form the path h, i. Then backtrack to h, and then to f. From f build the path f, d, e, c, a. Then backtrack to c and form the path c, b. This produces the spanning tree.

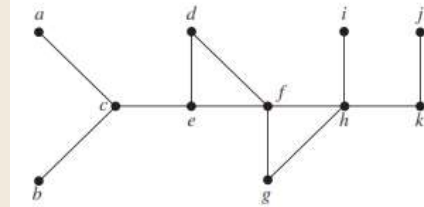


FIGURE 6 The Graph G.

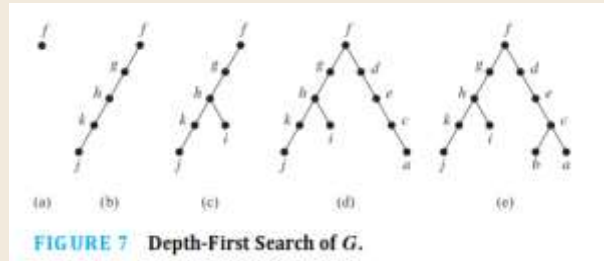


FIGURE 7 Depth-First Search of G.



Breadth-First Search



The **Breadth First Search (BFS)** algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before moving on to the nodes at the next depth level.

Breadth-First Traversal (or Search) for a graph is similar to the Breadth-First Traversal of a tree.

The only catch here is, that, unlike trees, graphs may **contain cycles**, so we may come to the same node again.



Breadth-First Search



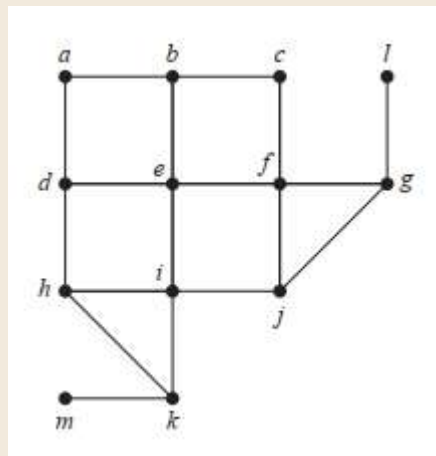
The algorithm works as follows:

1. Start by putting any one of the graph's vertices at the back of a **queue**.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones that aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

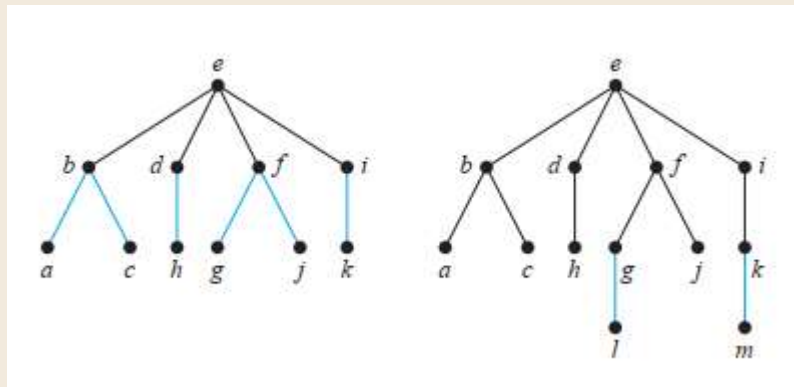
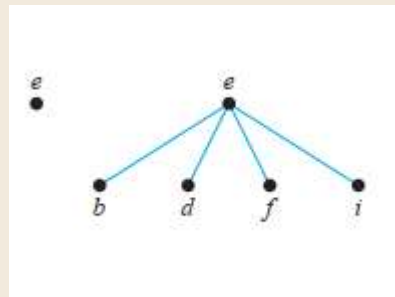
Breadth-First Search

Use breadth-first search to find a spanning tree for the following graph.

[Sp'22, Sp'19]



Ans:



Difference between DFS and BFS

#	Parameters	BFS	DFS
1.	Stands for	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
2.	Data Structure	BFS(Breadth First Search) uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure.
3.	Definition	BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level.	DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
4.	Technique	BFS can be used to find a single source shortest path in an unweighted graph because, in BFS, we reach a vertex with a minimum number of edges from a source vertex.	In DFS, we might traverse through more edges to reach a destination vertex from a source.
5.	Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
6.	Approach used	It works on the concept of FIFO (First In First Out).	It works on the concept of LIFO (Last In First Out).
7.	Suitable for	BFS is more suitable for searching vertices closer to the given source.	DFS is more suitable when there are solutions away from source.
8.	Suitability for Decision-Trees	BFS considers all neighbors first and therefore not suitable for decision-making trees used in games or puzzles.	DFS is more suitable for game or puzzle problems. We make a decision, and then explore all paths through this decision. And if this decision leads to win situation, we stop.
9.	Time Complexity	The Time complexity of BFS is $O(V + E)$ when Adjacency List is used and $O(V^2)$ when Adjacency Matrix is used, where V stands for vertices and E stands for edges.	The Time complexity of DFS is also $O(V + E)$ when Adjacency List is used and $O(V^2)$ when Adjacency Matrix is used, where V stands for vertices and E stands for edges.
10.	Visiting of Siblings/ Children	Here, siblings are visited before the children.	Here, children are visited before the siblings.

6

Minimum Spanning Tree



Minimum Spanning Tree



Define **Minimum Spanning Tree**. [Sp'19,'23, Aut'18]

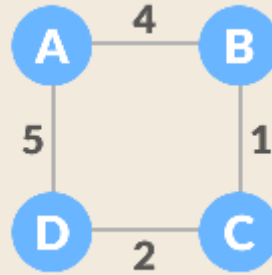
Ans: A minimum spanning tree is a spanning tree in which the sum of the weight of the edges is as minimum as possible.

The minimum spanning tree from a graph is found using the following algorithms:

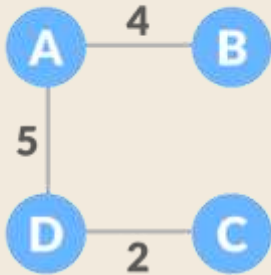
1. **Prim's** Algorithm
2. **Kruskal's** Algorithm

Minimum Spanning Tree

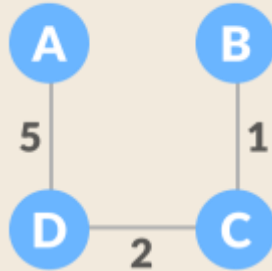
Find the minimum spanning tree of the following weighted graph:



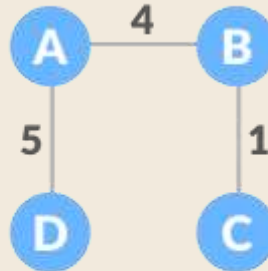
Ans: Some spanning trees of the graph are:



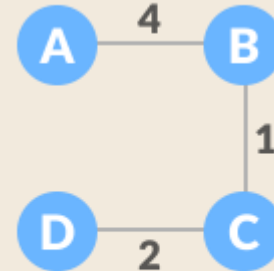
sum = 11



sum = 8



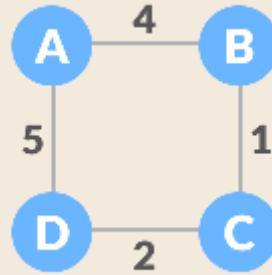
sum = 10



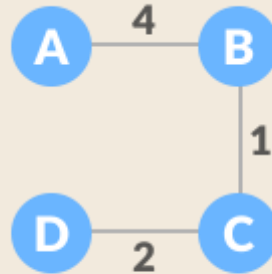
sum = 7

Minimum Spanning Tree

Find the minimum spanning tree of the following weighted graph:



Ans: So the minimum spanning tree of the graph is:

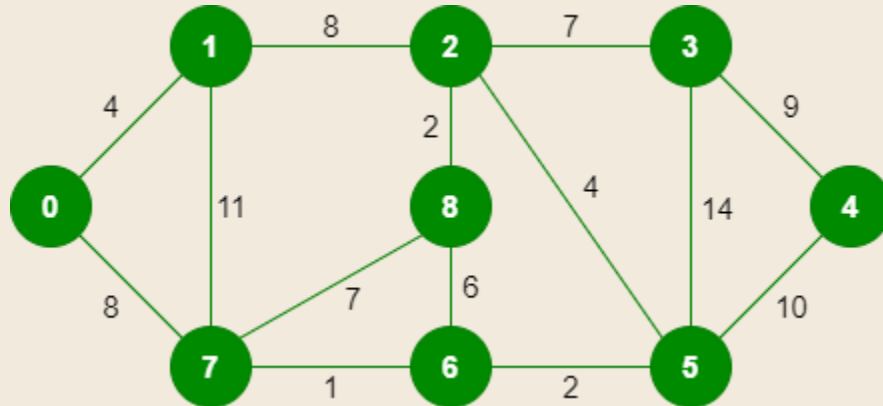


sum = 7

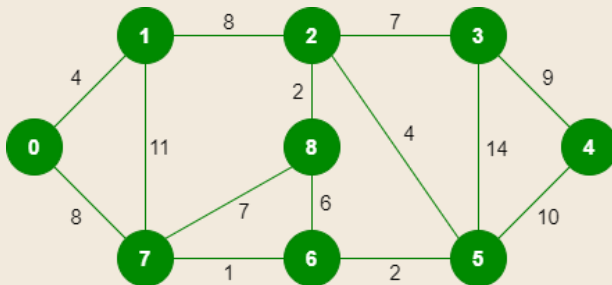
Kruskal's Minimum Spanning Tree

Kruskal's Minimum Spanning Tree (MST) Algorithm:

Kruskal's algorithm to find the minimum cost spanning tree uses the **greedy** approach. The Greedy Choice is to pick the **smallest** weight edge that does not cause a **cycle** in the MST constructed so far. Let us understand it with an example:



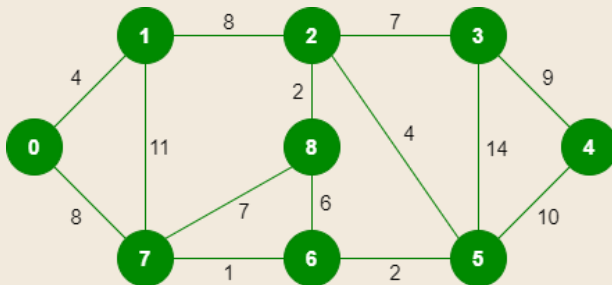
Kruskal's Minimum Spanning Tree



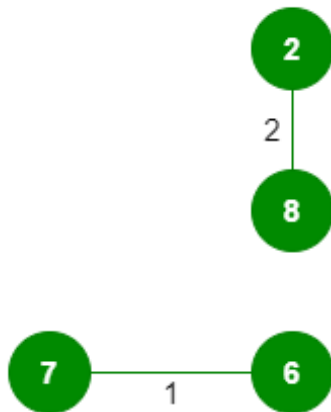
Step 1: Pick edge 7-6. No cycle is formed, include it.



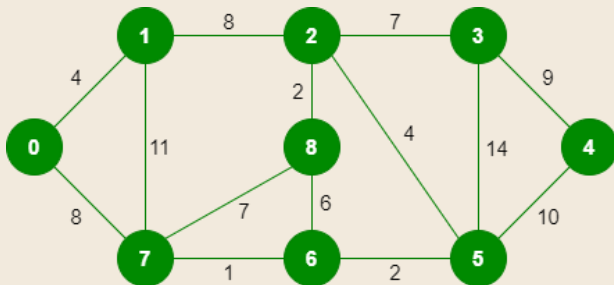
Kruskal's Minimum Spanning Tree



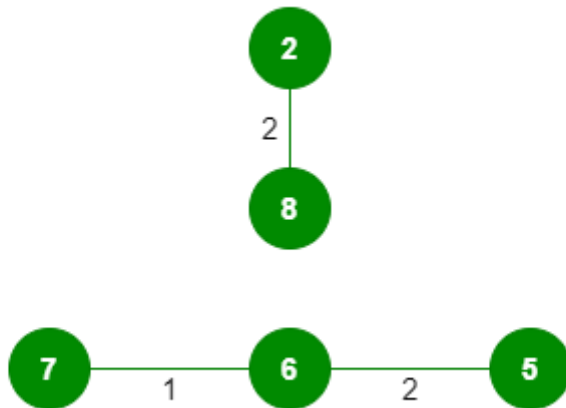
Step 2: Pick edge 8-2. No cycle is formed, include it.



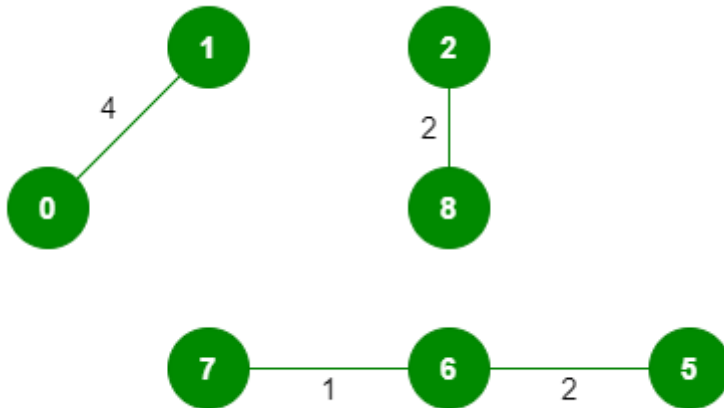
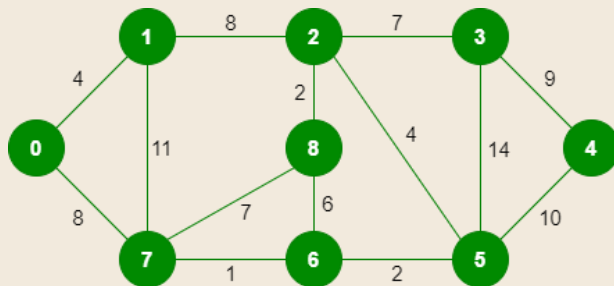
Kruskal's Minimum Spanning Tree



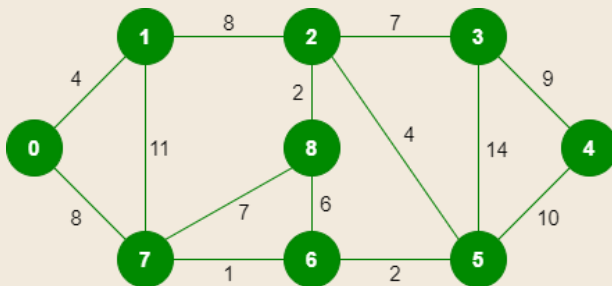
Step 3: Pick edge 6-5. No cycle is formed, include it.



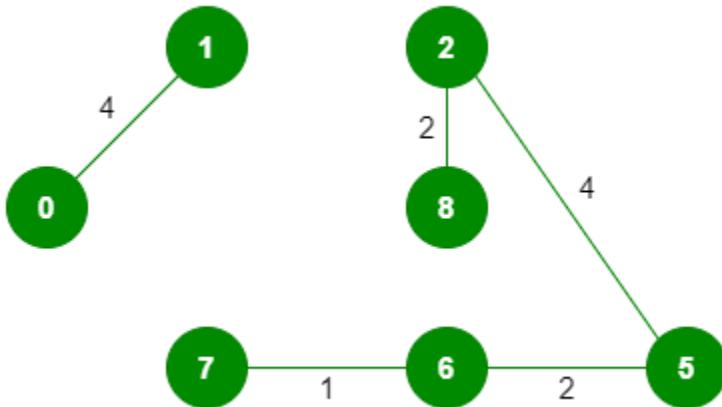
Kruskal's Minimum Spanning Tree



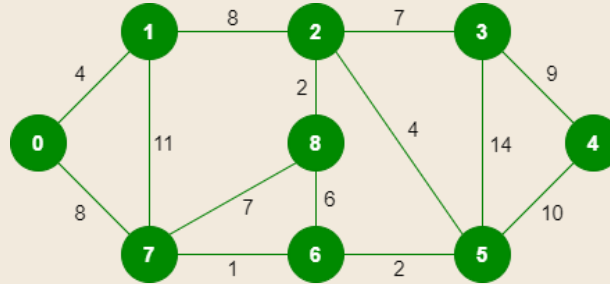
Kruskal's Minimum Spanning Tree



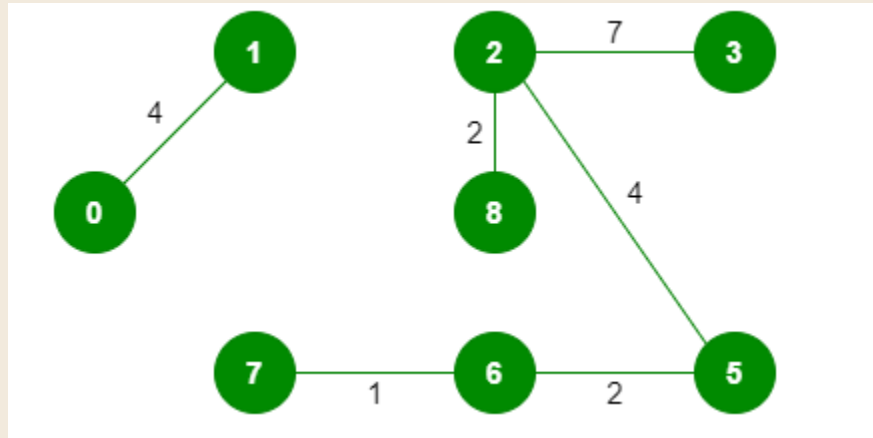
Step 5: Pick edge 2-5. No cycle is formed, include it.



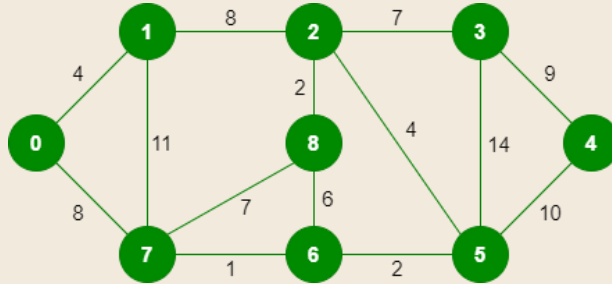
Kruskal's Minimum Spanning Tree



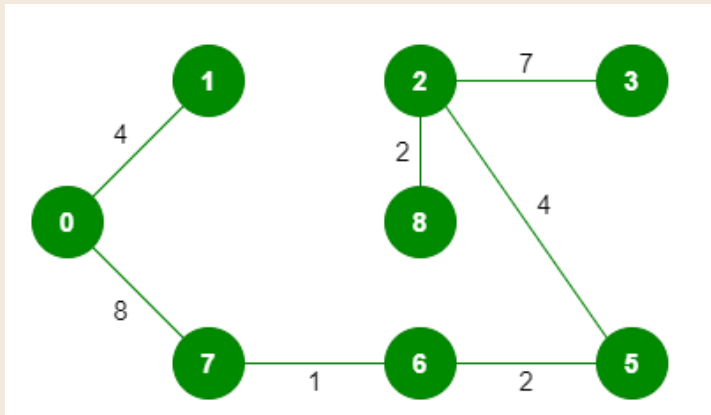
Step 6: Pick edge 8-6. Since including this edge results in the cycle, discard it. Pick edge 2-3: No cycle is formed, include it.



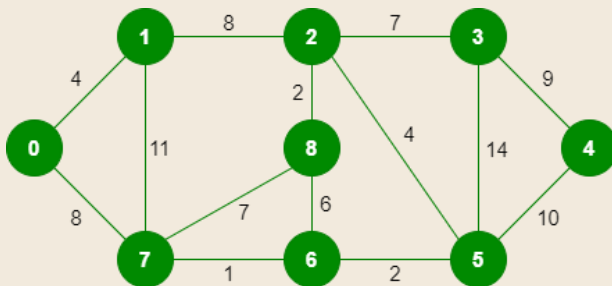
Kruskal's Minimum Spanning Tree



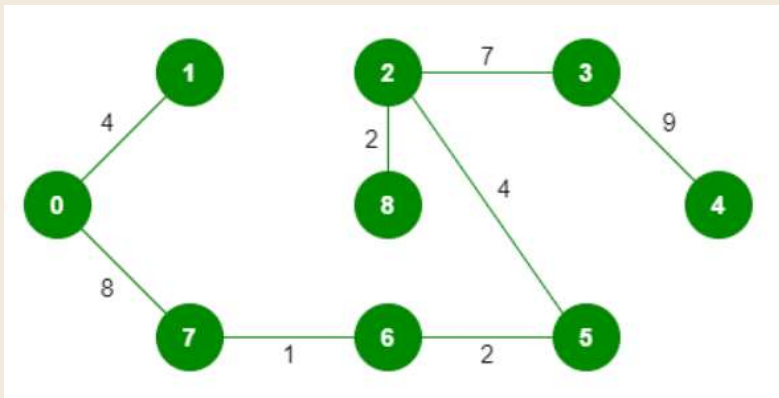
Step 7: Pick edge 7-8. Since including this edge results in the cycle, discard it. Pick edge 0-7. No cycle is formed, include it.



Kruskal's Minimum Spanning Tree



Step 8: Pick edge 1-2. Since including this edge results in the cycle, discard it. Pick edge 3-4. No cycle is formed, include it.

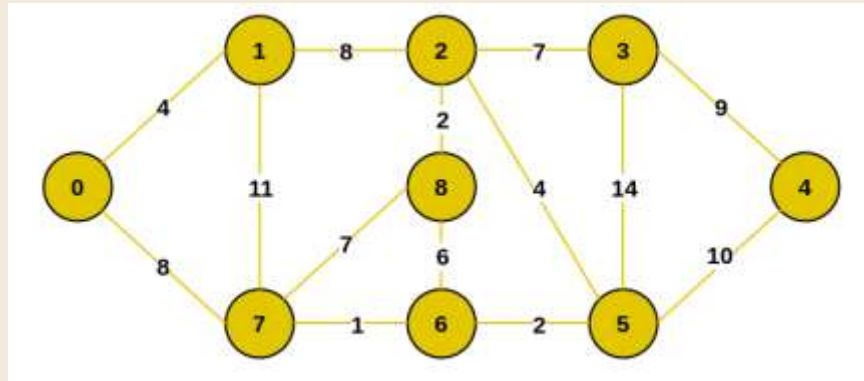


Note: Since the number of edges included in the MST equals to $(V - 1)$, so the algorithm stops here

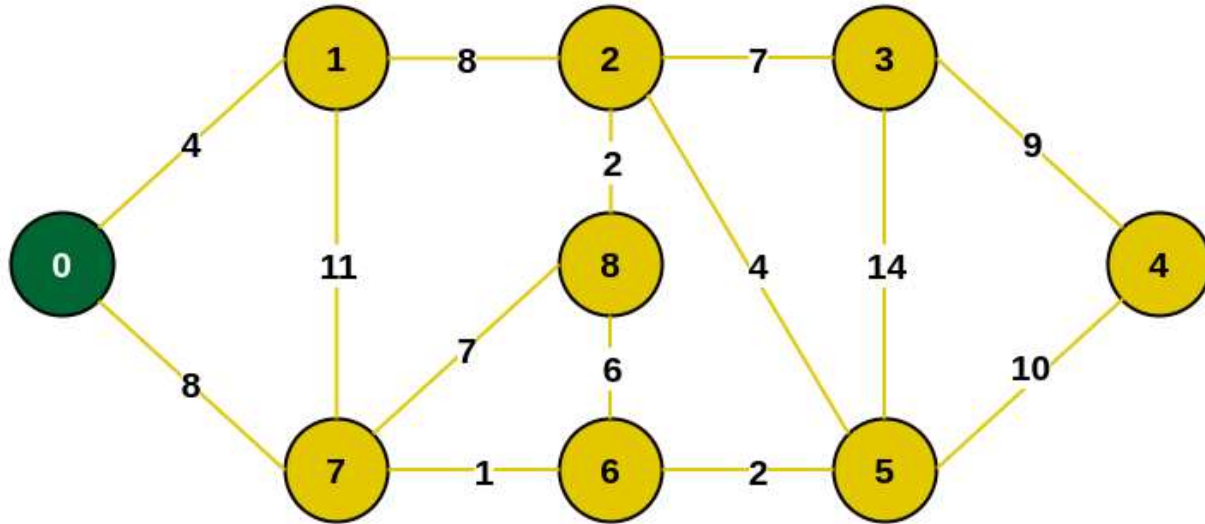
Prim's Minimum Spanning Tree

Prim's Minimum Spanning Tree (MST) Algorithm:

Prim's Algorithm also use Greedy approach to find the minimum spanning tree. In Prim's Algorithm we grow the spanning tree from a **starting position**. Unlike an edge in Kruskal's, we add vertex to the growing spanning tree in Prim's.

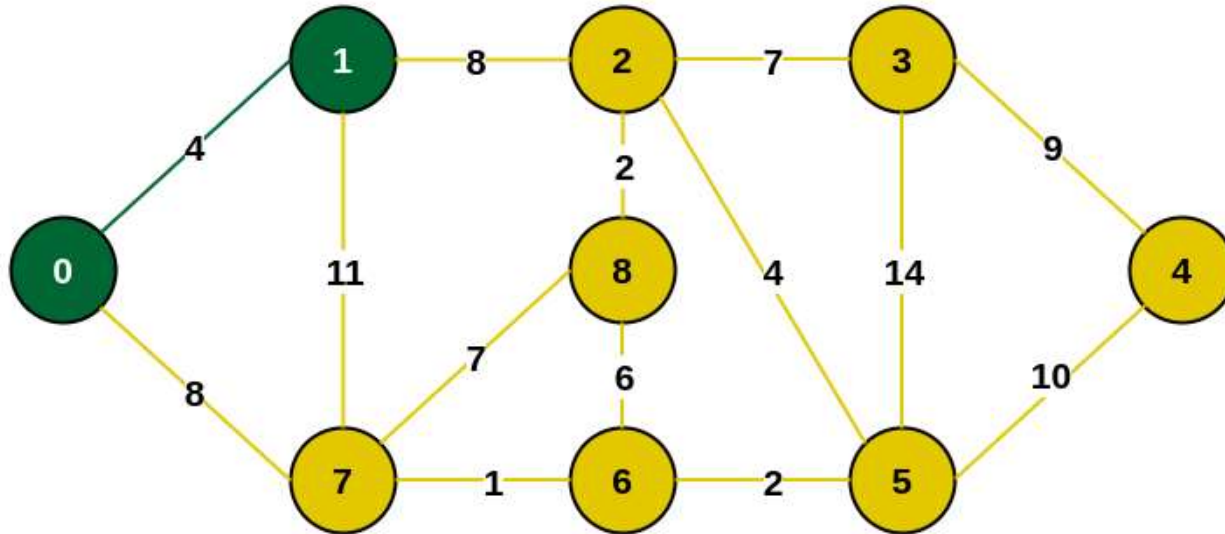


Prim's Minimum Spanning Tree



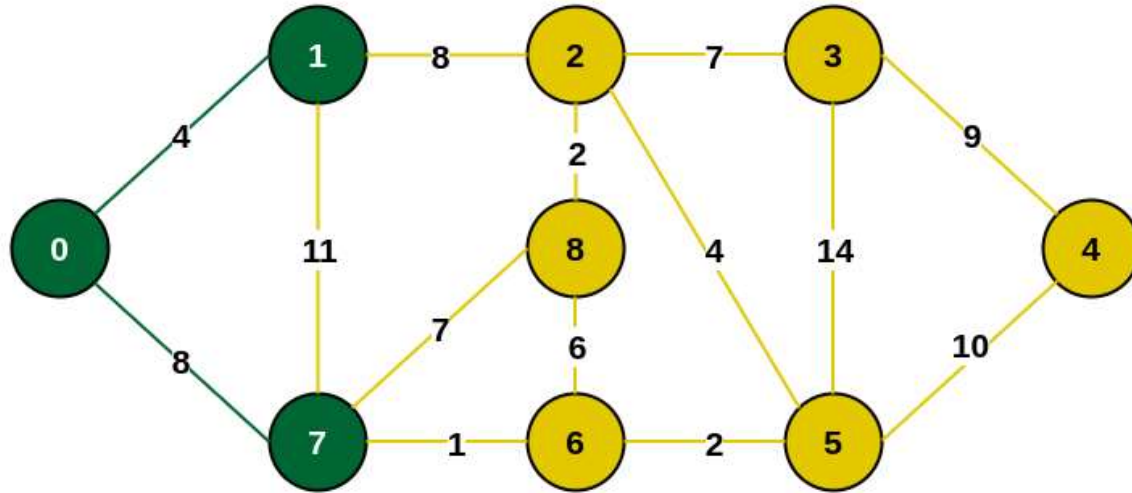
Select an arbitrary starting vertex. Here we have selected 0

Prim's Minimum Spanning Tree



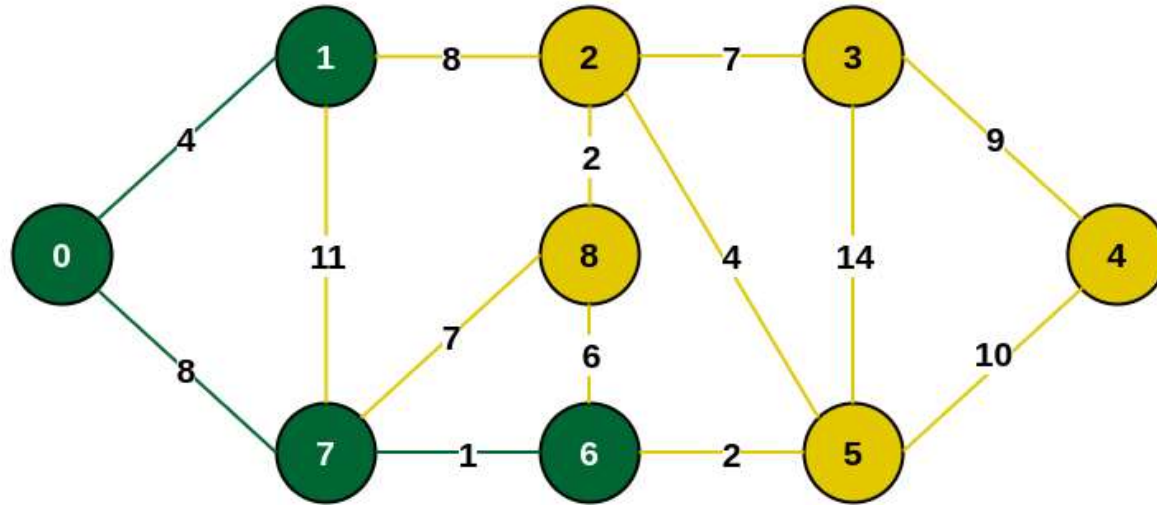
Minimum weighted edge from MST to other vertices is 0-1 with weight 4

Prim's Minimum Spanning Tree



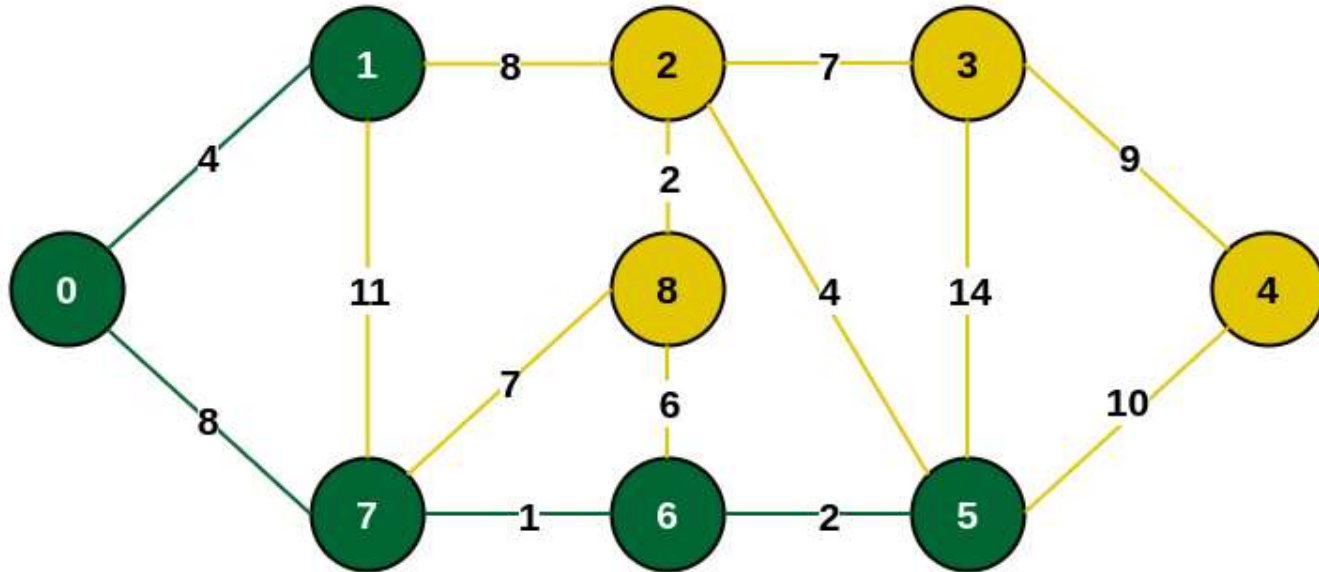
Minimum weighted edge from MST to other vertices is 0-7 with weight 8

Prim's Minimum Spanning Tree



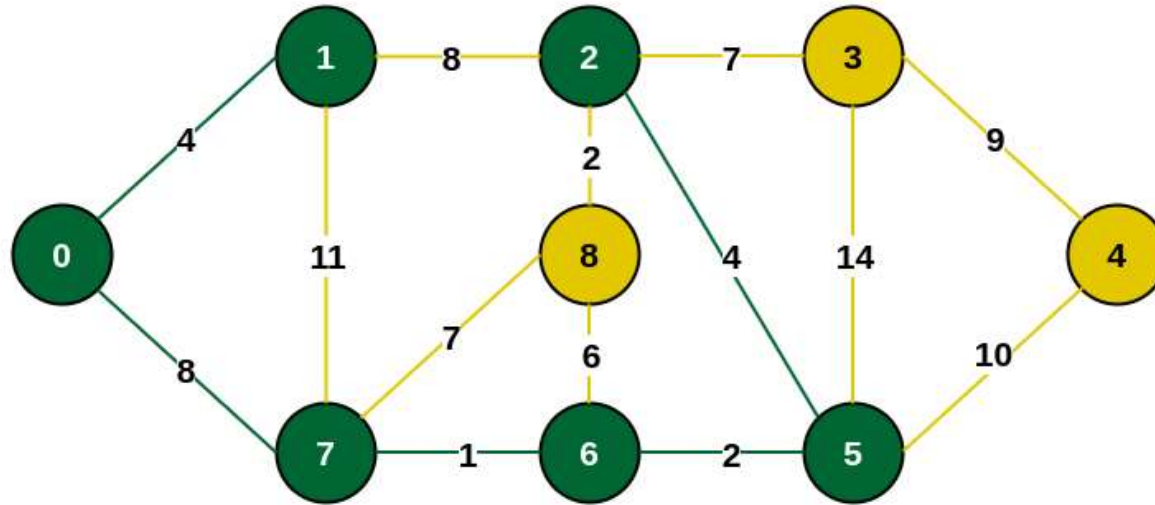
Minimum weighted edge from MST to other vertices is 7-6 with weight 1

Prim's Minimum Spanning Tree



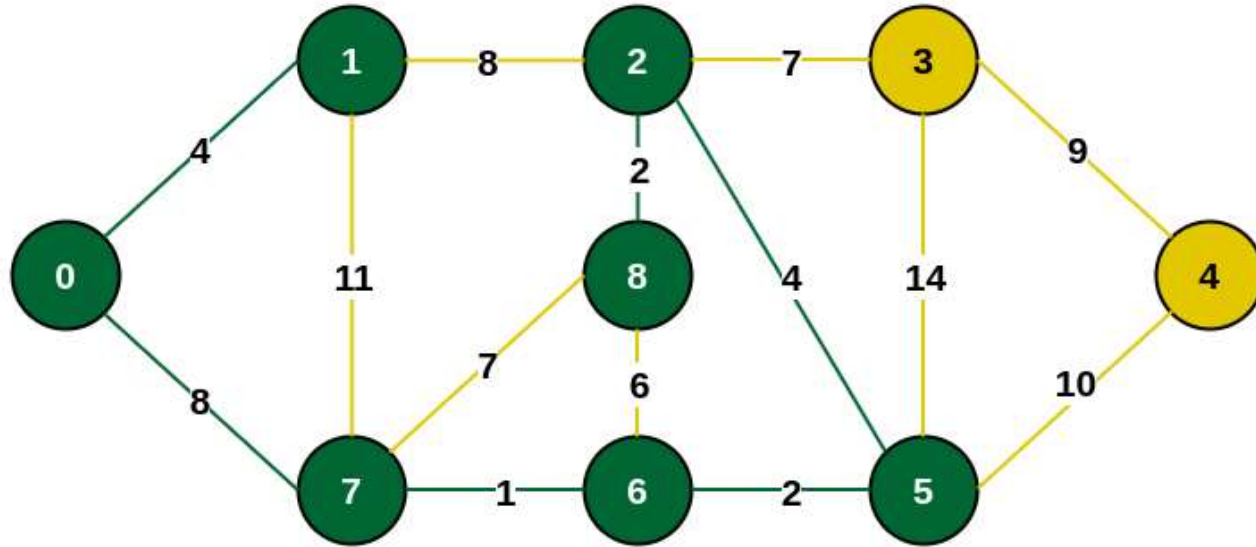
Minimum weighted edge from MST to other vertices is 6-5 with weight 2

Prim's Minimum Spanning Tree



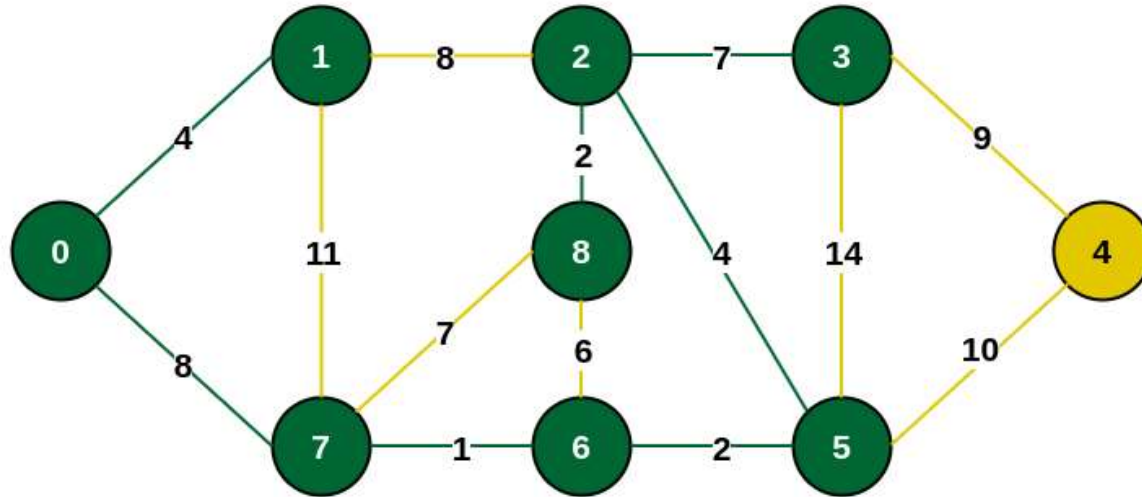
Minimum weighted edge from MST to other vertices is 5-2 with weight 4

Prim's Minimum Spanning Tree



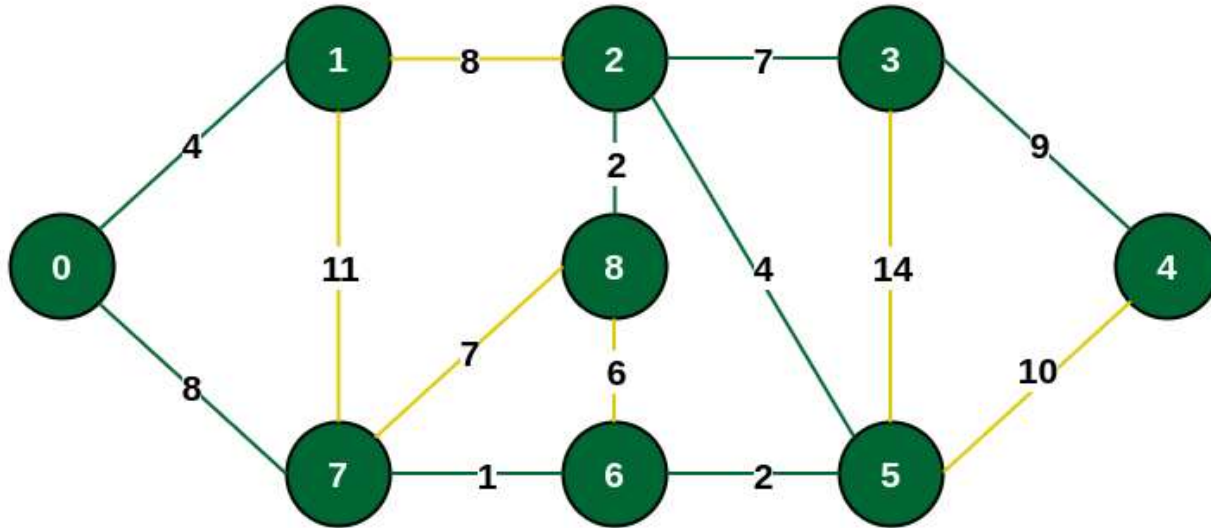
Minimum weighted edge from MST to other vertices is 2-8 with weight 2

Prim's Minimum Spanning Tree



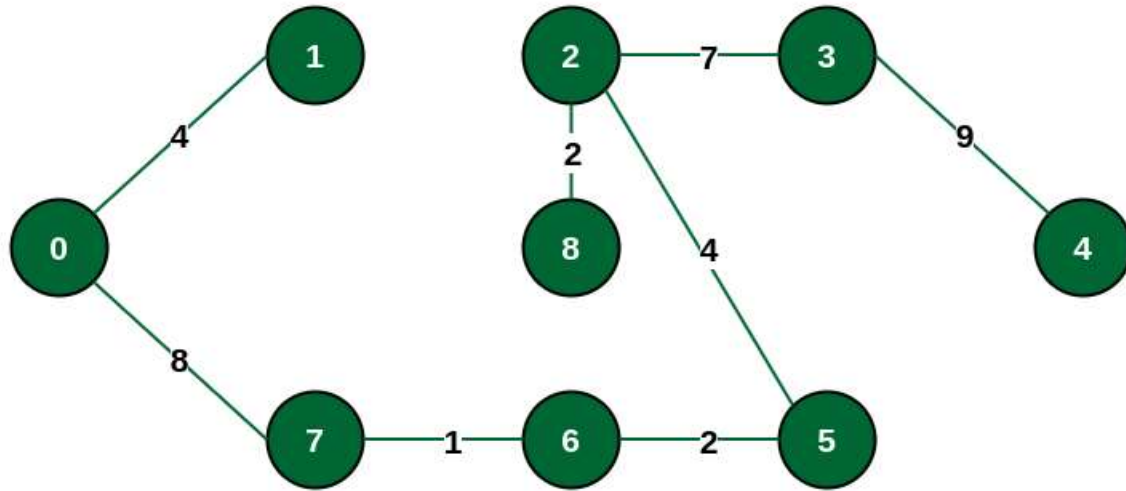
Minimum weighted edge from MST to other vertices is 2-3 with weight 7

Prim's Minimum Spanning Tree



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

Prim's Minimum Spanning Tree



The final structure of MST



Differences between prim's and kruskal algorithms



Write down the differences between prim's and kruskal algorithms with example. [Aut'22]

Ans:

Prim's Algorithm	Kruskal's Algorithm
It starts to build the Minimum Spanning Tree from any vertex in the graph.	It starts to build the Minimum Spanning Tree from the vertex carrying minimum weight in the graph.
It traverses one node more than one time to get the minimum distance.	It traverses one node only once.
Prim's algorithm has a time complexity of $O(V^2)$, V being the number of vertices and can be improved up to $O(E \log V)$ using Fibonacci heaps.	Kruskal's algorithm's time complexity is $O(E \log V)$, V being the number of vertices.
Prim's algorithm gives connected component as well as it works only on connected graph.	Kruskal's algorithm can generate forest(disconnected components) at any instant as well as it can work on disconnected components
Prim's algorithm runs faster in dense graphs.	Kruskal's algorithm runs faster in sparse graphs.
It generates the minimum spanning tree starting from the root vertex.	It generates the minimum spanning tree starting from the least weighted edge.
Applications of prim's algorithm are Travelling Salesman Problem, Network for roads and Rail tracks connecting all the cities etc.	Applications of Kruskal algorithm are LAN connection, TV Network etc.
Prim's algorithm prefer list data structures.	Kruskal's algorithm prefer heap data structures.

Minimum Spanning Tree Exercise

Exercise:

- Find the minimum spanning tree of the following graph. [Sp'22, Sp'19]
- Find the minimum spanning tree of the following graph. [Aut'18]
- Find the minimum spanning tree of the following graph. [Sp'23]

